

Parte A

1) Prompt enviado a Claude:

Diseña visualmente un esquema relacional utilizando entidades, atributos, tipos de datos, claves primarias y foraneas, sobre los siguientes requerimientos de un cliente:

1. Torneos. La organización realiza múltiples torneos. Cada torneo tiene un nombre, el título del videojuego que se juega, fechas de inicio y fin, y un pozo de premios (prize pool) en USD. Además tiene un número máximo de equipos por participar.
2. Jugadores. Los jugadores se registran en la plataforma con un gamertag (único), su nombre real, email, fecha de nacimiento y país de origen.
3. Equipos. Los jugadores forman equipos (de un número no acotado de jugadores). Cada equipo tiene un nombre único, una fecha de creación y un capitán. El capitán debe ser miembro del equipo. Un jugador pertenece a exactamente un equipo (la membresía es fija, no cambia entre torneos).
4. Inscripción a torneos. Los equipos se inscriben en torneos. Cada torneo tiene un número máximo de equipos que pueden participar. Un equipo puede inscribirse en múltiples torneos.
5. Partidas. Dentro de un torneo se juegan partidas (matches). Cada partida enfrenta a exactamente dos equipos distintos, tiene una fecha y hora programada, un resultado (puntaje equipo A y puntaje equipo B) y una fase del torneo. Las fases posibles son: fase de grupos, cuartos de final, semifinal, final. El formato del torneo es: 8 equipos divididos en 2 grupos de 4, round-robin dentro de cada grupo, los 2 mejores de cada grupo avanzan a semifinales, luego final.
6. Estadísticas individuales. Para cada partida se registran las estadísticas de cada jugador de los equipos participantes: KOs, restarts y assists.
7. Sponsors. Empresas pueden auspiciar torneos. De cada sponsor se conoce su nombre, industria (tecnología, bebidas, ropa, etc.) y el monto en USD que aporta a cada torneo que auspicia. Un sponsor puede auspiciar múltiples torneos.

(Esto se utilizará para crear una base de datos)

1.1) Respuesta de Claude:

Aquí tienes el esquema relacional visualizado con mermaid.js, mostrando todas las entidades, atributos, tipos de datos, y relaciones con sus respectivas claves primarias y foráneas.



Aquí está el esquema completo. A continuación un resumen de las decisiones de diseño más relevantes:

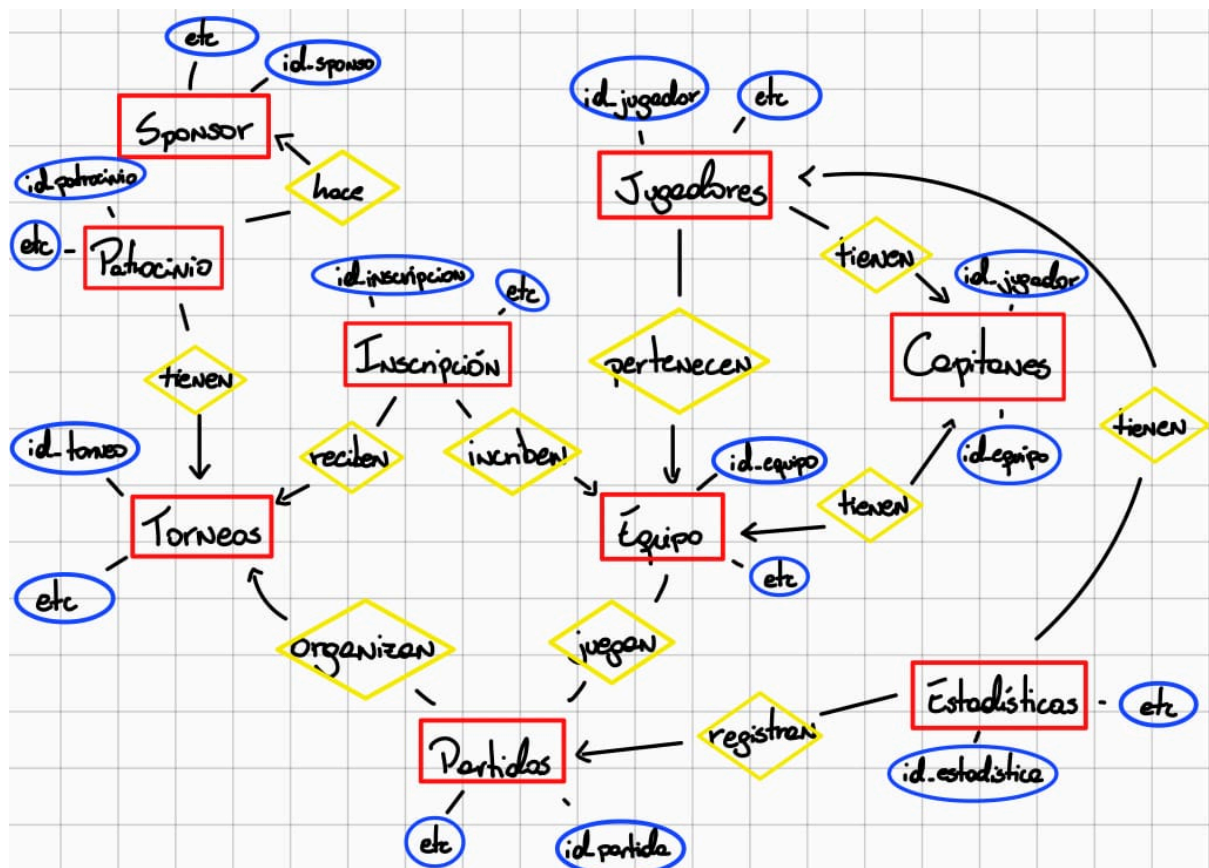
Entidades y sus roles:

TORNEO, JUGADOR, EQUIPO, SPONSOR son las entidades principales. INSCRIPCION, PATROCINIO y ESTADISTICA_JUGADOR son tablas de intersección que resuelven relaciones N:M.

Decisiones clave:

- EQUIPO tiene una FK `id_capitan` que apunta a JUGADOR. Esta referencia circular (JUGADOR tiene `id_equipo` FK → EQUIPO, y EQUIPO tiene `id_capitan` FK → JUGADOR) requiere manejo especial al insertar: primero se crea el equipo sin capitán, luego el jugador, luego se actualiza el capitán.
- PARTIDA tiene dos FK al equipo (`id_equipo_a`, `id_equipo_b`) para modelar que siempre se enfrentan exactamente dos equipos distintos. El campo `fase` es un ENUM con los valores: 'grupos', 'cuartos', 'semifinal', 'final'.
- ESTADISTICA_JUGADOR registra KOs, restarts y assists por jugador por partida, con (`id_partida`, `id_jugador`) como clave única candidata (puede hacerse compuesta).
- PATROCINIO es la tabla que permite que un SPONSOR auspicie múltiples torneos, guardando el monto por cada relación específica.
- Los campos `gamertag` y `email` en JUGADOR, y `nombre` en EQUIPO, tienen restricción UNIQUE según los requerimientos.

2) Esquema hecho por nosotros:



3) Análisis comparativo:

Al contrastar los modelos individuales con el esquema generado por Claude, se evidencia que, si bien la IA es capaz de identificar los componentes generales del sistema, presenta

omisiones estructurales importantes. El modelo automatizado integra entidades base como Equipo, Jugador, Partida, Estadística, Patrocinio, Sponsor, Torneo e Inscripción, pero falla al no incluir la entidad o el rol de Capitán. Esta entidad sirve para que la relación entre Jugador y Equipo funcione sin anomalías, y su ausencia no es solo un detalle conceptual, sino un error de arquitectura que compromete la funcionalidad técnica de la base de datos.

El principal problema derivado de esta omisión es la creación de una dependencia cíclica irresoluble en el modelo de la IA. Bajo su lógica, no sería posible registrar un equipo sin un ID de capitán válido, pero a su vez, no se podría dar de alta al jugador que ejercerá como capitán sin que este posea un ID de equipo previo. Este bloqueo operativo impediría la inserción inicial de datos, un conflicto de integridad que el diseño realizado por nosotros sí logra resolver. Nuestra propuesta considera estas relaciones de dependencia, asegurando que el flujo de creación de registros no se vea interrumpido por restricciones de llaves foráneas mal planteadas.

En conclusión, este ejercicio demuestra que la inteligencia artificial funciona como una herramienta valiosa para generar borradores rápidos, pero carece de la profundidad necesaria para capturar todas las restricciones de integridad y requerimientos de un cliente. La supervisión humana resulta, por tanto, un factor indispensable en el proceso ya que es capaz de darse cuenta y solucionar problemas de lógica relacional, garantizando que la base de datos sea una estructura funcional, coherente y respetando las restricciones de integridad.